

VIDEOJUEGO DODGER EN ESP32

Jailler Dany Gaviria Valencia

Laura Uribe

Daniela Diaz

Seguimiento IV - Electrónica Digital II

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Descripción del Sistema	2
4. Componentes del Sistema	3
4.1. Entradas	3
4.2. Procesamiento	3
4.3. Salidas	3
5. Modos de Juego	3
6. Lógica del Juego	3
7. Sistema de Sonido	4
8. Código del Programa	4
9. Funcionamiento del Sistema	11
10. Resultados	12
11. Conclusiones	12

1. Introducción

En este proyecto se implementa un videojuego tipo **Dodger** en un ESP32, utilizando una pantalla OLED SSD1306, botones físicos y un buzzer. El juego consiste en controlar un personaje que debe esquivar obstáculos en movimiento, mientras se incrementa progresivamente la dificultad.

El sistema incluye animaciones, efectos de sonido, música de fondo y múltiples modos de juego, lo que demuestra el uso avanzado de periféricos del ESP32.

2. Objetivos

2.1. Objetivo General

Diseñar e implementar un videojuego interactivo en tiempo real utilizando el microcontrolador ESP32.

2.2. Objetivos Específicos

- Implementar gráficos en una pantalla OLED SSD1306.
- Controlar el movimiento del jugador mediante botones.
- Generar obstáculos aleatorios en tiempo real.
- Detectar colisiones entre el jugador y los obstáculos.
- Implementar múltiples modos de juego.
- Integrar efectos de sonido y música con un buzzer.
- Diseñar un sistema de puntuación y récord.
- Implementar estados del juego (menú, juego, pausa y game over).

3. Descripción del Sistema

El sistema consiste en un videojuego donde el jugador controla un sprite que se mueve verticalmente para evitar colisionar con obstáculos.

El juego se ejecuta en un ESP32 y utiliza:

- Pantalla OLED para visualización gráfica
- Botones para control del jugador
- Buzzer para efectos de sonido y música
- Generación aleatoria de obstáculos

El sistema cuenta con diferentes modos de juego que modifican la dificultad y la dinámica.

4. Componentes del Sistema

4.1. Entradas

- Botón de movimiento hacia arriba
- Botón de movimiento hacia abajo
- Botón de inicio/pausa

4.2. Procesamiento

- Microcontrolador ESP32
- Lógica del juego en tiempo real
- Generación de obstáculos aleatorios
- Control de animaciones
- Manejo de estados del sistema

4.3. Salidas

- Pantalla OLED (gráficos del juego)
- Buzzer (sonidos y música)
- Visualización de puntaje y récord

5. Modos de Juego

El sistema incluye tres modos:

- **Hardcore:** Aumenta rápidamente la dificultad.
- **Tiempo:** El juego tiene un límite de tiempo.
- **Clásico:** Modo estándar sin restricciones.

6. Lógica del Juego

El sistema funciona mediante una máquina de estados:

- **MENÚ:** Selección de modo de juego.
- **JUEGO:** Ejecución principal.
- **PAUSA:** Detiene temporalmente el juego.

- **GAME OVER:** Finalización del juego.

Durante el juego:

- El jugador se mueve verticalmente.
- Los obstáculos se desplazan horizontalmente.
- La velocidad aumenta con el puntaje.
- Se detectan colisiones en tiempo real.

7. Sistema de Sonido

El sistema utiliza un buzzer controlado por PWM para generar:

- Efectos de movimiento y colisión
- Sonido de inicio y pausa
- Música de fondo tipo Mario

La música se genera mediante una secuencia de frecuencias y duraciones.

8. Código del Programa

El siguiente código implementa el videojuego completo incluyendo control de hardware, lógica del juego, animaciones y sonido.

```

1 from machine import Pin, I2C, PWM
2 import ssd1306, framebuf, time, random
3
4 # ----- HARDWARE -----
5 i2c = I2C(0, scl=Pin(22), sda=Pin(21), freq=400000)
6 oled = ssd1306.SSD1306_I2C(128, 64, i2c)
7
8 if not hasattr(oled, "blit"):
9     oled.blit = oled.framebuf.blit
10 if not hasattr(oled, "fill_rect"):
11     oled.fill_rect = oled.framebuf.fill_rect
12
13 oled.fill(0)
14 oled.text("INICIANDO...", 10, 25)
15 oled.show()
16
17 # ----- BOTONES -----
18 btn_up = Pin(32, Pin.IN, Pin.PULL_UP)
19 btn_down = Pin(33, Pin.IN, Pin.PULL_UP)

```

```

20 btn_start = Pin(4, Pin.IN, Pin.PULL_UP)
21
22 prev_up = 1
23 prev_down = 1
24 prev_start = 1
25
26 last_move_time = 0
27 move_delay = 120
28
29 # ----- BUZZER -----
30 buzzer = PWM(Pin(25))
31 buzzer.duty_u16(0)
32
33 def tono(freq, dur):
34     buzzer.freq(freq)
35     buzzer.duty_u16(60000)
36     time.sleep(dur)
37     buzzer.duty_u16(0)
38
39 def parar_musica():
40     buzzer.duty_u16(0)
41
42 def fx(freq, dur):
43     parar_musica()
44     tono(freq, dur)
45
46 # ----- SONIDOS -----
47 def sonido_punto(): fx(1200,0.02)
48 def sonido_inicio(): fx(800,0.05); fx(1200,0.05)
49 def sonido_pausa(): fx(500,0.05)
50 def sonido_reanudar(): fx(1000,0.05)
51 def sonido_muerte(): fx(400,0.08); fx(200,0.12)
52 def sonido_gameover(): fx(600,0.08); fx(400,0.08); fx(200,0.15)
53
54 def sonido_modos(m):
55     if m==0: fx(600,0.02); fx(800,0.02)
56     elif m==1: fx(900,0.02); fx(1100,0.02)
57     elif m==2: fx(1200,0.02); fx(1500,0.02)
58
59 # ----- MUSICA TIPO MARIO -----
60 melodia = [
61     659,659,0,659,0,523,659,0,784,0,0,0,
62     392,0,0,0,
63     523,0,392,0,330,0,440,0,494,0,466,440,
64     392,659,784,880,698,784,659,523,587,494
65 ]
66
67 duraciones = [

```

```

68     120,120,120,120,120,120,120,120,200,120,120,120,
69     200,120,120,120,
70     120,120,120,120,120,120,120,120,120,120,120,
71     120,120,120,120,120,120,120,120,120,200
72 ]
73
74 indice_nota = 0
75 ultimo_tono = 0
76
77 def musica_fondo(now):
78     global indice_nota, ultimo_tono
79
80     if time.ticks_diff(now, ultimo_tono) > duraciones[indice_nota]:
81         ultimo_tono = now
82         nota = melodia[indice_nota]
83
84         if nota == 0:
85             buzzer.duty_u16(0)
86         else:
87             buzzer.freq(nota)
88             buzzer.duty_u16(10000)
89
90         indice_nota = (indice_nota + 1) % len(melodia)
91
92 # ----- SPRITES -----
93 sprite1 = bytearray([0x18,0x3C,0x7E,0xDB,0x7E,0x24,0x18,0x00])
94 sprite2 = bytearray([0x10,0x38,0x7C,0xFE,0x7C,0x28,0x10,0x00])
95
96 frame = 0
97 fb_jugador = framebuffer.FrameBuffer(sprite1, 8, 8, framebuffer.MONO_VLSB)
98
99 # ----- ESTADOS -----
100 MENU, JUEGO, PAUSA, GAME_OVER = 0,1,2,3
101 estado = MENU
102
103 # ----- VARIABLES -----
104 modos = ["Hardcore","Tiempo","Clasico"]
105 modo = 0
106 TIEMPO_LIMITE = 10
107
108 jugador_y = 30
109 obstaculos = []
110 obstaculos_extra = []
111 tipo_obs = 0
112
113 GAP_SIZE = 20
114 velocidad = 3
115

```

```

116 score = 0
117 high_score = 0
118 start_time = time.ticks_ms()
119
120 pausa_inicio = 0
121 gameover_sonando = False
122
123 # ANIMACIONES MENU
124 menu_anim_y = 30
125 menu_dir = 1
126 menu_timer = 0
127 blink = True
128 menu_arrow_offset = 0
129 menu_arrow_dir = 1
130
131 # ----- FUNCIONES -----
132 def centrar(txt,y):
133     oled.text(txt,(128-len(txt)*6)//2,y)
134
135 # ----- LOOP -----
136 while True:
137
138     now = time.ticks_ms()
139
140     up_now = btn_up.value()
141     down_now = btn_down.value()
142     start_now = btn_start.value()
143
144     up_pressed = (prev_up == 1 and up_now == 0)
145     down_pressed = (prev_down == 1 and down_now == 0)
146     start_pressed = (prev_start == 1 and start_now == 0)
147
148     prev_up = up_now
149     prev_down = down_now
150     prev_start = start_now
151
152     # ----- MENU -----
153     if estado==MENU:
154
155         parar_musica()
156
157         if up_pressed:
158             modo=(modo-1)%3
159             sonido_modo(modo)
160
161         if down_pressed:
162             modo=(modo+1)%3
163             sonido_modo(modo)

```

```

164
165     if start_pressed:
166         sonido_inicio()
167         estado=JUEGO
168         score=0
169         tipo_obs=0
170         gameover_sonando=False
171
172         obstaculos=[{"x":128,"gap_y":random.randint(10,40)},
173                     {"x":200,"gap_y":random.randint(10,40)}]
174
175         obstaculos_extra=[{"x":160,"y":random.randint(10,50)},
176                           {"x":260,"y":random.randint(10,50)}]
177
178         jugador_y=30
179         start_time=time.time()
180
181     if time.time()-start_time > 100:
182         start_time = time.time()
183         menu_anim_y += menu_dir
184         if menu_anim_y > 40 or menu_anim_y < 20:
185             menu_dir *= -1
186
187         blink = not blink
188
189         menu_arrow_offset += menu_arrow_dir
190         if menu_arrow_offset > 2 or menu_arrow_offset < -2:
191             menu_arrow_dir *= -1
192
193     frame = (frame + 1) % 20
194     fb_menu = framebuffer.FrameBuffer(
195         sprite1 if frame < 10 else sprite2, 8, 8, framebuffer.
196         MONO_VLSB
197     )
198
199     oled.fill(0)
200     centrar("DODGER",0)
201
202     #           MEN EN COLUMNA
203     for i, m in enumerate(modos):
204         y = 18 + i*12
205         x = 40
206
207         if i == modo:
208             oled.text(">", x-10 + menu_arrow_offset, y)
209
210         oled.text(m, x, y)

```



```

211     # ave animada
212     oled.blit(fb_menu, 90, menu_anim_y)
213
214     if blink:
215         centrar("START",55)
216
217     oled.show()
218
219     # ----- JUEGO -----
220     elif estado==JUEGO:
221
222         musica_fondo(now)
223
224         if start_pressed:
225             sonido_pausa()
226             pausa_inicio = now
227             estado=PAUSA
228             continue
229
230         frame = (frame + 1) % 20
231         fb_jugador = framebuf.FrameBuffer(
232             sprite1 if frame < 10 else sprite2, 8, 8, framebuf.
                MONO_VLSB
233         )
234
235         if now - last_move_time > move_delay:
236             if up_now == 0 and jugador_y > 0:
237                 jugador_y -= 3
238                 last_move_time = now
239             elif down_now == 0 and jugador_y < 56:
240                 jugador_y += 3
241                 last_move_time = now
242
243         velocidad = 3 + score//5
244         gap_actual = max(12, GAP_SIZE - score//5)
245
246         if tipo_obs == 0:
247             for obs in obstaculos:
248                 obs["x"] -= velocidad
249                 if obs["x"] < -8:
250                     obs["x"] = 128
251                     obs["gap_y"] = random.randint(10,40)
252                     score += 1
253                     sonido_punto()
254                     if score % 5 == 0:
255                         tipo_obs = 1
256         else:
257             for obs in obstaculos_extra:

```

```

258         obs["x"] -= velocidad + 1
259         if obs["x"] < -8:
260             obs["x"] = 128
261             obs["y"] = random.randint(5,55)
262             score += 1
263             sonido_punto()
264             if score % 5 == 0:
265                 tipo_obs = 0
266
267         tiempo = (now - start_time)//1000
268         tiempo_restante = TIEMPO_LIMITE - tiempo
269
270         if tipo_obs == 0:
271             for obs in obstaculos:
272                 if 10 < obs["x"] < 18:
273                     if not (obs["gap_y"] < jugador_y < obs["gap_y"] +
274                             gap_actual):
275                         sonido_muerte()
276                         estado=GAME_OVER
277             else:
278                 for obs in obstaculos_extra:
279                     if abs(obs["x"]-10)<6 and abs(obs["y"]-jugador_y)<6:
280                         sonido_muerte()
281                         estado=GAME_OVER
282
283         if estado==GAME_OVER:
284             if score > high_score:
285                 high_score = score
286             continue
287
288         if modo==1 and tiempo_restante <= 0:
289             sonido_muerte()
290             estado=GAME_OVER
291             if score > high_score:
292                 high_score = score
293             continue
294
295         oled.fill(0)
296         oled.blit(fb_jugador,10,jugador_y)
297
298         if tipo_obs == 0:
299             for obs in obstaculos:
300                 oled.fill_rect(obs["x"],0,8,obs["gap_y"],1)
301                 oled.fill_rect(obs["x"],obs["gap_y"]+gap_actual
302                                ,8,64-(obs["gap_y"]+gap_actual),1)
303             else:
304                 for obs in obstaculos_extra:
305                     oled.fill_rect(obs["x"],obs["y"],6,6,1)

```

```

304
305     oled.text("P:"+str(score),0,0)
306     oled.text("H:"+str(high_score),60,0)
307
308     if modo==1:
309         oled.text(str(max(0,tiempo_restante)),100,0)
310
311     oled.show()
312
313     # ----- PAUSA -----
314     elif estado==PAUSA:
315
316         parar_musica()
317
318         oled.fill(0)
319         centrar("PAUSA",25)
320         oled.show()
321
322         if start_pressed:
323             sonido_reanudar()
324             pausa_duracion = now - pausa_inicio
325             start_time += pausa_duracion
326             estado=JUEGO
327
328     # ----- GAME OVER -----
329     elif estado==GAME_OVER:
330
331         parar_musica()
332
333         if not gameover_sonando:
334             sonido_gameover()
335             gameover_sonando = True
336
337         oled.fill(0)
338         centrar("GAME OVER",5)
339         centrar("Score:"+str(score),25)
340         centrar("High:"+str(high_score),40)
341         oled.show()
342
343         if start_pressed:
344             estado=MENU

```

9. Funcionamiento del Sistema

El sistema inicia mostrando un menú donde el usuario puede seleccionar el modo de juego. Durante la ejecución:

- Se leen continuamente los botones
- Se actualiza la posición del jugador
- Se generan y mueven obstáculos
- Se detectan colisiones
- Se actualiza el puntaje
- Se reproducen sonidos y música

Si ocurre una colisión, el sistema cambia al estado de Game Over.

10. Resultados

El sistema logró ejecutar correctamente un videojuego en tiempo real con gráficos, sonido y control interactivo.

Se observó:

- Respuesta inmediata a los botones
- Animaciones fluidas en pantalla
- Incremento progresivo de dificultad
- Correcta detección de colisiones

11. Conclusiones

El proyecto demuestra la capacidad del microcontrolador para manejar gráficos, sonido y lógica compleja simultáneamente.

Además, el sistema es adaptable y puede mejorarse con nuevos modos de juego, gráficos o funcionalidades adicionales.